Indian Institute of Technology, Gandhinagar

CS - 327 Compilers

Lexical Specification of the C Language
Prof. Shouvick Mondal

Team:
Sailaja Mudu (23110288)
Vadithya Harsha Vardhan Nayak (23110349)
Rushitha Thipparapu (23110338)

MiniC Language Specification

➢ Introduction

This document presents the lexical specification of a simplified C-like programming language designed for the study of lexical analysis. The language follows the basic syntax and structure of C while intentionally limiting its features to focus on core lexical concepts. It supports essential constructs such as keywords, identifiers, integer literals, operators, and delimiters, while excluding advanced features of the full C language. The specification formally defines the character set, token categories, and rules for identifying valid and invalid tokens.

The selected language is a simplified C-like programming language that follows an imperative and procedural programming style. The language is case-sensitive, meaning that identifiers such as var, Var, and VAR are treated as distinct. Conceptually, the language is treated as a compiled language, where source code is analysed and translated before execution. Only a limited set of data types is supported to keep the language simple and suitable for lexical analysis. Whitespace characters such as spaces, tabs, and newlines are used only to separate tokens and improve readability. They do not affect the meaning of the program and are ignored during lexical analysis.

➢ Alphabet and Character Set (Σ)

The language uses a restricted ASCII-based character set. A source program is treated as a sequence of characters from this alphabet. Any character outside this set causes a lexical error. The character set of the language is based on the ASCII character encoding and consists of the following categories:

1. **Letters**
   The language allows both uppercase and lowercase English letters, specifically:
   - Lowercase letters: a to z
   - Uppercase letters: A to Z
   - The language is case-sensitive (var, Var, and VAR are different).
   - Used in identifiers and keywords.
2. **Digits**
   - 0–9
   - Used to form integer literals.
   - May appear in identifiers, but not as the first character.
3. **Special Symbols**
   - + - * / = < > ( ) { } ;
   - Represent operators and delimiters.
   - Each symbol is recognised as a separate token.
4. **Whitespace**
   a. Space, tab, newline
   b. Used only to separate tokens and improve readability.
   c. Ignored by the lexer.

Invalid Characters
Any character outside the above categories (e.g., @, #, $, non-ASCII) is invalid and produces a lexical error.

➢ Tokens and Lexical Units

This section introduces the concept of **tokens** and describes how the lexical analyser identifies and classifies tokens from the source program.

Supported Tokens in MiniC:

- **Keywords**

Keywords are reserved identifiers that have special meaning in MiniC. They cannot be used as variable, function, or type names. MiniC recognises the following keywords:

Category	Keywords
Type Specifiers	Int, long, short, float, double ,char ,void
Control Flow	If, else, while, do, for, switch ,case, default
Jump Statements	break, continue,return
User-Defined	struct
Operators	sizeof

- **Identifiers**

An identifier is a sequence of characters used to name variables, functions, and other user-defined entities

Formation Rules:

1) Must begin with a letter (a-z, A-Z) or underscore (_)
2) Subsequent characters can be letters, digits (0-9), or underscores
3) Case-sensitive (count, Count, and COUNT are different identifiers)
4) Cannot be a reserved keyword
5) No length limit (though reasonable length is recommended

Regular Expression : [a-zA-Z_][a-zA-Z0-9_]*

Valid Examples: x, count, _temp, studentAge, MAX_SIZE, var123
Invalid Examples: 123var (starts with digit), my-var (contains hyphen), int (keyword)

- **Constants and Literals**
  1. **Integer Constants**

     Integer constants can be specified in decimal or hexadecimal notation

     a. **Decimal :** Sequence of digits (0-9)

Examples: 0, 42, 1234, 999
Regex: [0-9]+

b. **Hexadecimal** : Prefix '0x' followed by hex digits (0-9, a-f)
Examples: 0x1A, 0xff, 0x1234
Regex: 0x[0-9a-fA-F]+ or 0[xX][0-9a-fA-F]+
Note: Upper-case 'X' (0X...) is also supported

2. **Floating-Point Constants**
Floating-point constants represent real numbers. •
Format: Digits, decimal point, digits, optional exponent
Examples: 3.14, 0.5, 2.0, 1.5e10, 3.2E-5
Regex: [0-9]+\.[0-9]+([eE][+-]?[0-9]+)?

3. **Character Constants**
A character constant is a single character enclosed in single quotes
Format: Single character or escape sequence in single quotes
Examples: 'a', 'Z', '5', '\n', '\t', '\\'
Escape sequences: \n (newline), \t (tab), \\ (backslash), \' (single quote)

- **String Literals**
String literals are sequences of characters enclosed in double quotes. MiniC supports both single strings and multiple adjacent strings that are concatenated.

Single String:
"Hello, World!"
"This is a string with \n newline"

Multiple Concatenated Strings:
"First part " "second part" → treated as "First part second part"

Escape sequences in strings: \n, \t, \\, \", \r

- **Operators**

Category	**Operators**	**Description**
Arithmetic	+ - * / %	Addition, Subtraction, Multiplication, Division, Modulo
Relational	< > <= >= == !=	Less than, Greater than, Less/Equal, Greater/Equal, Equal, Not Equal
Logical	&& || !	Logical AND, Logical OR, Logical NOT
Bitwise	& | ^ ~	Bitwise AND, OR, XOR, NOT

Assignment	=	Simple Assignment
Increment/Decrement	++ --	Post/Pre increment, Post/Pre decrement
Array/Function	[ ] ( )	Array subscript, Function call
Type	sizeof (type)	Size of type, Typecast
Three-way Comparison	<=>	Returns -1, 0, or 1

Three-way Comparison Operator (<=>):
This operator is a MiniC extension not present in standard C. It compares two values and returns:

- -1 if the left operand is less than the right operand
- 0 if operands are equal
- 1 if the left operand is greater than the right operand

Example: result = a <=> b;

- **Punctuators and Delimiters**

Symbol	**Usage**
;	Statement terminator
,	Separator in declarations, expressions, and argument lists
( )	Function calls, function parameters, expression grouping
{ }	Compound statements (code blocks)
[ ]	Array subscripting, array declarations
:	Case labels in switch statements

- **Comments**

MiniC supports two styles of comments, both inherited from C :

Single-line Comments :
Begin with // and continue to the end of the line.
// This is a single-line comment

Multi-line Comments:
Begin with /* and end with */. Can span multiple lines.
/* This is a multi-line comment */

Important: Multi-line comments must be properly closed. An unclosed comment (missing */) results in a lexical error. Comments do not nest.

- **Invalid / Error Tokens**
  Any sequence of characters that does not match the rules of the defined token categories is classified as an invalid token. These tokens indicate lexical errors and are reported by the lexer.

➢ Features NOT Supported in MiniC

Category	**Unsupported Features**
Preprocessor	#include, #define, #ifdef, #ifndef, etc
Operators	Ternary (?:), Compound assignments (+=, -=, *=, /=, %=, &=, ^=, |=), Shift operators (<<, >>)
Keywords	auto, const, goto, inline, register, restrict, signed, static, typedef, unsigned, enum, union, volatile
Advanced Types	Function pointers, Variadic functions (...), Unions, Enumerations
Other	__func__, Designated initialisers, Flexible array members